Jhanvi Cars — Local to Kubernetes Migration

Complete migration runbook: local development → Docker → GitHub → Helm → AWS kubeadm Kubernetes

Scope. This document records the actual Jhanvi Cars migration work performed in this project, including architecture, containerization, Helm, Secrets, PostgreSQL, Memcached, deployment, troubleshooting, and the remaining migration-runtime issue. Secret values are intentionally omitted.

1. Target architecture

The application is a microservices platform deployed into the **jhanvi-cars** Kubernetes namespace. Backend services communicate through Kubernetes DNS rather than localhost. PostgreSQL is shared by the services using separate schemas and application roles; Memcached is used by vehicle-service.

```
API Gateway       :3000
Auth service      :3001
User service      :3002
Driver service    :3003
Booking service   :3004
Vehicle service   :3005
Payment service   :3006
Notification      :3007
Frontend          :80
PostgreSQL        :5432
Memcached         :11211
```

2. Migration flow

```
Ubuntu development VM
        |
        v
Git feature branch
        |
        v
Docker images -> Docker Hub
        |
        v
Helm chart -> GitHub
        |
        v
AWS kubeadm master
  git fetch / pull
        |
        v
helm upgrade --install
        |
        v
Kubernetes
  Namespace / ConfigMap / Secret
  PostgreSQL + PVC / Memcached
  Services + Deployments
  Migration Jobs / Ingress
        |
        v
Smoke test application
```

3. Git workflow

The migration was developed on `feature/kubernetes-migration`, keeping `main` as the stable source of truth. The final database-permission fix was committed and pushed as `233496b`.

```
# Development VM
cd ~/jhanvi-dev/helm/jhanvi-cars
git status
git branch --show-current

# Validate
helm lint .
helm template jhanvi-cars . >/tmp/jhanvi-cars-rendered.yaml

# Publish
git add templates/postgres-init.yaml
git commit -m "fix: allow service users to create schema objects"
git push origin feature/kubernetes-migration
```

```
# AWS master: synchronize explicitly
cd ~/jhanvi-cars-k8s
git fetch origin
git pull --ff-only origin feature/kubernetes-migration
git log -1 --oneline
git status
```

4. Containerization

All backend services have multi-stage Node 20 Alpine Dockerfiles: build first, then a runtime image. The frontend builds Vite and serves `dist` using Nginx.

```
jhanviadmin/jhanvi-cars-api-gateway:1.0
jhanviadmin/jhanvi-cars-auth-service:1.0
jhanviadmin/jhanvi-cars-user-service:1.0
jhanviadmin/jhanvi-cars-driver-service:1.0
jhanviadmin/jhanvi-cars-booking-service:1.0
jhanviadmin/jhanvi-cars-vehicle-service:1.0
jhanviadmin/jhanvi-cars-payment-service:1.0
jhanviadmin/jhanvi-cars-notification-service:1.0
jhanviadmin/jhanvi-cars-frontend:latest
```

The frontend image was built using BuildKit so `.env.local` was supplied as a build secret rather than copied into the image build context as a normal file:

```
DOCKER_BUILDKIT=1 docker build \
  --secret id=frontend_env,src=.env.local \
  -t jhanviadmin/jhanvi-cars-frontend:latest .
```

5. Helm chart

Helm v3.22.0 is used. The chart is `helm/jhanvi-cars`.

```
helm/jhanvi-cars/
■■■ Chart.yaml
■■■ values.yaml
■■■ templates/
    ■■■ _helpers.tpl
    ■■■ configmap.yaml
    ■■■ deployment.yaml
    ■■■ frontend.yaml
    ■■■ ingress.yaml
    ■■■ memcached.yaml
    ■■■ migrations.yaml
    ■■■ namespace.yaml
    ■■■ postgres-init.yaml
    ■■■ postgres.yaml
    ■■■ service.yaml
```

Validation succeeded with Helm lint. Template rendering was also successful. The rendered PostgreSQL grants were checked after adding CREATE privilege.

```
helm lint .
helm template jhanvi-cars . >/tmp/jhanvi-cars-rendered.yaml
grep 'GRANT USAGE' /tmp/jhanvi-cars-rendered.yaml
```

6. Kubernetes configuration

Local service URLs using localhost were replaced for Kubernetes with ClusterIP service DNS names. Memcached similarly uses the service name `memcached`.

```
AUTH_SERVICE_URL=http://auth-service:3001
USER_SERVICE_URL=http://user-service:3002
DRIVER_SERVICE_URL=http://driver-service:3003
BOOKING_SERVICE_URL=http://booking-service:3004
VEHICLE_SERVICE_URL=http://vehicle-service:3005
PAYMENT_SERVICE_URL=http://payment-service:3006
NOTIFICATION_SERVICE_URL=http://notification-service:3007
MEMCACHED_HOST=memcached
MEMCACHED_PORT=11211
MEMCACHED_TTL_SECONDS=60
PROXY_TIMEOUT_MS=5000
NODE_ENV=production
```

7. Kubernetes Secrets

The cluster uses `jhanvi-cars-secrets`. It contains database credentials and application secrets such as JWT/internal-service/Razorpay values. Secret values are not stored in the Git repository.

```
Database URLs:
DATABASE_URL=postgresql://user_user:<password>@postgres:5432/jhanvi_cars
AUTH_DATABASE_URL=postgresql://auth_user:<password>@postgres:5432/jhanvi_cars

Secret keys used:
POSTGRES_PASSWORD
USER_DB_PASSWORD
AUTH_DB_PASSWORD
DATABASE_URL
AUTH_DATABASE_URL
INTERNAL_SERVICE_TOKEN
JWT_SECRET
RAZORPAY_KEY_ID
RAZORPAY_KEY_SECRET
```

For this practice cluster, the Secret was created/updated with kubectl and the temporary credential files used during setup were removed afterward.

8. PostgreSQL migration design

PostgreSQL runs as a StatefulSet with a persistent volume. Database name is `jhanvi_cars`. The application roles are `user_user` and `auth_user`.

```
Schemas:
auth_schema
user_schema
driver_schema
booking_schema
vehicle_schema
payment_schema
notification_schema
```

A ConfigMap-mounted initialization script creates the extension, schemas, roles, and privileges. The service migrations subsequently create/update tables.

```
GRANT USAGE, CREATE ON SCHEMA user_schema TO user_user;
GRANT USAGE, CREATE ON SCHEMA driver_schema TO user_user;
GRANT USAGE, CREATE ON SCHEMA booking_schema TO user_user;
GRANT USAGE, CREATE ON SCHEMA vehicle_schema TO user_user;
GRANT USAGE, CREATE ON SCHEMA payment_schema TO user_user;
GRANT USAGE, CREATE ON SCHEMA notification_schema TO user_user;
GRANT USAGE, CREATE ON SCHEMA auth_schema TO auth_user;
```

Important: PostgreSQL's official initialization process executes init scripts only when the data directory is initialized. Updating the ConfigMap does not automatically rerun initialization against an already initialized PVC.

9. First Helm deployment issue

The namespace already existed but lacked Helm ownership metadata. Helm therefore refused to import it into the release.

```
kubectl label namespace jhanvi-cars   app.kubernetes.io/managed-by=Helm --overwrite

kubectl annotate namespace jhanvi-cars   meta.helm.sh/release-name=jhanvi-cars   meta.helm.sh/release-namespace=jhanvi-cars
```

After adoption, the release installed successfully as revision 1.

10. Deploy the chart

```
cd ~/jhanvi-cars-k8s

helm upgrade --install jhanvi-cars ./helm/jhanvi-cars   --namespace jhanvi-cars   --create-namespace
```

Successful result recorded: `STATUS: deployed`, `REVISION: 1`.

11. Initial Kubernetes verification

```
kubectl get all -n jhanvi-cars
kubectl get pvc -n jhanvi-cars
kubectl get ingress -n jhanvi-cars
kubectl get jobs -n jhanvi-cars
```

During startup, PostgreSQL and Memcached initially showed ContainerCreating. They later became 1/1 Running. The application Deployments eventually reached 1/1 as well. The remaining issue was with database migration Jobs.

12. Migration Jobs

Each migration Job waits for PostgreSQL and runs the service's existing `npm run migrate`. The current wait container uses BusyBox and `nc -z postgres 5432`.

```
kubectl get jobs -n jhanvi-cars
kubectl describe job user-migration -n jhanvi-cars
```

Several Jobs reached BackoffLimitExceeded. Their original pods were deleted, so direct Job logs timed out. Cluster events confirmed the migration containers started and then failed.

13. Reproducing a migration failure

A temporary debug pod was created with the same service image, ConfigMap, Secret, schema, and migration command. This exposed the actual application error instead of only Kubernetes BackOff events.

```
kubectl logs user-migration-debug -n jhanvi-cars
```

Observed output:

```
> user-service@1.0.0 migrate
> knex --knexfile knexfile.ts migrate:latest

Failed to load external module ts-node/register
Failed to load external module typescript-node/register
...
Unexpected token '{'
SyntaxError: Unexpected token '{'
```

Root cause identified: the production migration process points Knex at `knexfile.ts`, but the production image does not have a working TypeScript runtime/loader for that configuration. The failure occurs before the intended database migration can complete.

14. Correct next fix

Do not patch PostgreSQL credentials based on this error. Fix the service image/migration packaging consistently across the services.

```
# Inspect on development VM
cd ~/jhanvi-dev

cat backend/user-service/package.json
cat backend/user-service/Dockerfile
cat backend/user-service/knexfile.ts
```

For a compiled production image, a clean approach is to compile/use a JavaScript Knex configuration and compiled migration files, or explicitly include the required TypeScript runtime if that is the chosen architecture. After validating user-service, apply the same verified pattern to auth, driver, booking, vehicle, payment, and notification migrations.

15. Final verification checklist

```
# Helm
helm list -n jhanvi-cars
helm status jhanvi-cars -n jhanvi-cars

# Workloads
kubectl get pods -n jhanvi-cars
kubectl get deployments -n jhanvi-cars
kubectl get statefulsets -n jhanvi-cars

# Networking
kubectl get svc -n jhanvi-cars
kubectl get ingress -n jhanvi-cars

# Storage
kubectl get pvc -n jhanvi-cars

# Database migrations
kubectl get jobs -n jhanvi-cars

# Troubleshooting
```

```
kubectl describe pod <pod> -n jhanvi-cars
kubectl logs <pod> -n jhanvi-cars
kubectl describe job <job> -n jhanvi-cars
```

16. Application smoke test

1. Frontend loads through Ingress
2. /api reaches API Gateway
3. API Gateway resolves backend services through Kubernetes DNS
4. User registration/login works
5. Driver registration and vehicle selection work
6. Vehicle availability works
7. Booking creation works
8. Booking data persists in PostgreSQL
9. Razorpay payment flow works
10. Notification flow works
11. Vehicle-service Memcached caching works

17. Production-style lessons from this migration

• Separate application configuration from Secrets.
• Never assume localhost works after containerization; use service discovery.
• Validate Helm with both `helm lint` and `helm template`.
• Treat database initialization and migrations as separate concerns.
• Keep migration Jobs observable and rerunnable.
• Explicitly synchronize deployment environments with Git before deploying.
• Debug the actual container command rather than guessing from Kubernetes status alone.

18. Current migration status

Completed: Dockerization, Docker Hub images, Helm chart, Helm validation, Kubernetes Secret setup, PostgreSQL StatefulSet/PVC, PostgreSQL initialization, Memcached, backend/frontend Deployments, ClusterIP Services, Ingress configuration, Git feature-branch workflow, AWS synchronization, and successful Helm installation.

Remaining: repair the migration runtime packaging/configuration so Knex can load migrations in the production images; rerun all migration Jobs; confirm schemas/tables; then perform the full frontend → gateway → services → PostgreSQL/payment/notification smoke test.

This status reflects the recorded project state at document creation. It intentionally does not claim that the migration is fully complete because the migration Jobs were still failing.